

LAPORAN PRAKTIKUM 4
PEMOGRAMAN BERORIENTASI OBJEK
EKSPANSI PRODUK KEUANGAN (*INHERITANCE*)



Disusun Oleh:
Annisa Layli Ramadhani
2511532024

Dosen Pengampu:
Nurfiah, S.St, M.Kom

Asisten Praktikum:
Muhammad Dawi Syauqi

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

A. Langkah-Langkah

1. Memastikan Akses Modifier di Rekening.java

a. Buat Class Rekening

```
private String nomorRekening;
```

Membuat atribut *nomorRekening* dengan tipe data *String* untuk menyimpan nomor rekening. *Modifier private* digunakan agar atribut tidak dapat diakses secara langsung dari luar *class*.

```
private String namaPemilik;
```

Membuat atribut *namaPemilik* dengan tipe data *String* untuk menyimpan nama pemilik rekening.

```
private String pin;
```

Membuat atribut *pin* dengan tipe data *String* untuk menyimpan PIN rekening. *Modifier private* digunakan agar PIN tidak dapat diakses secara langsung dari luar *class*.

b. Protected untuk mengakses Subclass langsung

```
protected double saldo;
```

Membuat atribut *saldo* dengan tipe data *double* untuk menyimpan jumlah saldo rekening. *Modifier protected* memungkinkan atribut diakses oleh *class* turunan.

```
protected ArrayList<Transaksi_2511532024> riwayatTransaksi;
```

Membuat *ArrayList* bernama *riwayatTransaksi* untuk menyimpan banyak objek *Transaksi_2511532024* sebagai riwayat transaksi rekening.

2. Membuat Subclass RekeningTabungan

```
public class RekeningTabungan extends Rekening_2511532024 {
```

Membuat class *RekeningTabungan* yang merupakan turunan dari class *Rekening_2511532024* menggunakan keyword *extends*.

```
private double sukuBunga;
```

Membuat atribut *sukuBunga* dengan tipe data *double* untuk menyimpan *persentase* bunga rekening tabungan.

```
public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
```

Membuat *constructor* *RekeningTabungan* untuk memberikan nilai awal berupa nomor rekening, nama pemilik, saldo awal, PIN, dan suku bunga.

```
super(nomor, nama, saldoAwal, pinAwal);
```

Memanggil *constructor* dari class induk *Rekening_2511532024* untuk menginisialisasi data nomor rekening, nama, saldo awal, dan PIN. Modul menjelaskan bahwa *super()* digunakan untuk memanggil *constructor* induk.

```
this.sukuBunga = sukuBunga;
```

Memasukkan nilai parameter *sukuBunga* ke dalam atribut *sukuBunga* milik objek.

```
public void tambahBungaAkhirBulan() {
```

Membuat method *tambahBungaAkhirBulan()* untuk menjalankan proses penambahan bunga pada rekening tabungan.

```
double nominalBunga = saldo * (sukuBunga / 100);
```

Menghitung nominal bunga berdasarkan saldo rekening dan persentase suku bunga

```
saldo += nominalBunga;
```

Menambahkan nominal bunga yang diperoleh ke dalam saldo

rekening.

```
String idTrx = "TRX-B-" + System.currentTimeMillis();
```

Membuat *ID* transaksi bunga dengan menggabungkan teks "TRX-B-" dan waktu saat transaksi dibuat.

```
riwayatTransaksi.add(new Transaksi_2511532024(idTrx,  
"Bunga", nominalBunga));
```

Menambahkan transaksi bunga ke dalam *riwayatTransaksi* sebagai catatan transaksi rekening.

```
System.out.println("Bunga " + sukuBunga + "% berhasil  
ditambahkan: Rp" + nominalBunga);
```

Menampilkan informasi bahwa bunga berhasil ditambahkan beserta persentase dan nominal bunganya.

3. Membuat Subclass RekeningGiro

```
public class RekeningGiro extends Rekening_2511532024 {
```

Baris ini digunakan untuk membuat *class* baru bernama *RekeningGiro*. Kata *extends* menunjukkan bahwa *RekeningGiro* merupakan turunan dari *class Rekening_2511532024*. Artinya, *RekeningGiro* dapat mewarisi atribut dan *method* yang dimiliki oleh *Rekening_2511532024*.

```
private double batasOverdraft;
```

Baris ini membuat atribut bernama *batasOverdraft* dengan tipe data *double*. Atribut tersebut digunakan untuk menyimpan nilai batas *overdraft* pada rekening giro. Kata *private* berarti atribut ini hanya dapat diakses secara langsung dari dalam *class RekeningGiro*.

```
public RekeningGiro(String nomor, String nama, double
```

```
saldoAwal, String pinAwal, double batasOverdraft) {
```

Baris ini merupakan constructor dari class *RekeningGiro*. *Constructor* digunakan untuk memberikan nilai awal ketika sebuah objek *RekeningGiro* dibuat. *Constructor* ini menerima beberapa parameter, yaitu nomor untuk nomor rekening, nama untuk nama pemilik, *saldoAwal* untuk saldo awal, *pinAwal* untuk *PIN*, dan *batasOverdraft* untuk menentukan batas overdraft rekening giro.

```
super(nomor, nama, saldoAwal, pinAwal);
```

Baris ini digunakan untuk memanggil *constructor* dari class *Rekening_2511532024* sebagai *superclass*. Nilai nomor, nama, *saldoAwal*, dan *pinAwal* dikirim ke *constructor superclass* agar data dasar rekening dapat diinisialisasi oleh class induknya. Dengan begitu, *RekeningGiro* tidak perlu menulis ulang proses inisialisasi data rekening dasar.

```
this.batasOverdraft = batasOverdraft;
```

Baris ini digunakan untuk memasukkan nilai parameter *batasOverdraft* ke atribut *batasOverdraft* milik objek *RekeningGiro*. *this.batasOverdraft* mengacu pada atribut yang berada di dalam class, sedangkan *batasOverdraft* yang berada setelah tanda sama dengan merupakan parameter yang diterima oleh *constructor*.

```
public double getBatasOverdraft() {
```

Baris ini membuat *method* bernama *getBatasOverdraft()*. *Method* tersebut bersifat *public*, sehingga dapat dipanggil dari luar class. Tipe *double* menunjukkan bahwa *method* ini akan mengembalikan nilai berupa angka desimal, yaitu nilai dari batas overdraft.

```
return batasOverdraft;
```

Baris ini mengembalikan nilai yang tersimpan di dalam atribut

batasOverdraft. Jadi, ketika *method getBatasOverdraft()* dipanggil, program akan mengambil dan memberikan nilai batas *overdraft* rekening tersebut.

B. Tugas/Latihan Praktikum

1. Pembuatan Multi-Produk

Modifikasi case 1: buka Rekening Baru di Main.java

- Setelah user menginput saldo awal dan PIN, berikan opsi produk: “Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis”
- Jika user memilih tabungan, minta input suku bunga dalam persen, lalu instansiasi objek *RekeningTabungan*.
- Jika user memilih Giro, minta input *batasOverdraft* (limit pinjaman), lalu instansiasi objek *RekeningGiro*.
- Simpan objek tersebut ke dalam *daftarRekening* (Upcasting: Subclass otomatis dikenali sebagai tipe Superclassnya).

a. Class Main.

```
System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis | 3. Rekening VIP");
System.out.print("Pilihan: ");
int produk = input.nextInt();
input.nextLine();

Rekening_2511532024 rekeningBaru;
if (produk == 1) {
    System.out.print("Suku Bunga (%): ");
    double sukuBunga = input.nextDouble();
    input.nextLine();
    rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
} else if (produk == 2) {
    System.out.print("Masukkan Batas Overdraft: ");
    double batasOverdraft = input.nextDouble();
    input.nextLine();
    rekeningBaru = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
} else {
    System.out.println("Pilihan produk tidak valid!");
    break;
}

daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
```

```
System.out.println("Rekening berhasil ditambahkan.");  
break;
```

Penjelasan Code:

a. Class Main

```
System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro  
Bisnis");
```

Baris ini menampilkan pilihan jenis produk rekening yang dapat dipilih oleh pengguna. Pengguna diberikan tiga pilihan, yaitu 1 untuk Tabungan Umum, 2 untuk Giro Bisnis

```
System.out.print("Pilihan: ");
```

Baris ini menampilkan tulisan "Pilihan: " tanpa membuat baris baru. Program kemudian menunggu pengguna memasukkan nomor produk yang ingin dipilih.

```
int produk = input.nextInt();
```

Baris ini digunakan untuk membaca angka yang dimasukkan oleh pengguna menggunakan `nextInt()`. Angka tersebut disimpan ke dalam variabel `produk` yang bertipe `int`. Nilai inilah yang nantinya digunakan untuk menentukan jenis rekening yang akan dibuat.

```
input.nextLine();
```

Baris ini digunakan untuk mengambil sisa karakter *newline* (*Enter*) yang masih tersimpan setelah penggunaan `nextInt()`. Ini dilakukan agar pembacaan input berikutnya menggunakan `nextLine()` tidak langsung dilewati oleh program.

```
Rekening_2511532024 rekeningBaru;
```

Baris ini mendeklarasikan sebuah variabel bernama `rekeningBaru` dengan tipe `Rekening_2511532024`. Variabel ini nantinya digunakan untuk menyimpan objek rekening yang dibuat, baik objek `RekeningTabungan` maupun `RekeningGiro`. Hal ini bisa

dilakukan karena kedua *class* tersebut merupakan turunan dari *Rekening_2511532024*.

```
if (produk == 1) {
```

Baris ini melakukan pengecekan apakah pengguna memilih produk dengan nomor 1. Jika nilai produk sama dengan 1, maka program akan menjalankan kode di dalam blok *if* untuk membuat rekening tabungan.

```
System.out.print("Suku Bunga (%): ");
```

Jika pengguna memilih produk 1, baris ini meminta pengguna memasukkan nilai suku bunga rekening tabungan dalam bentuk persen.

```
double sukuBunga = input.nextDouble();
```

Baris ini membaca nilai suku bunga yang dimasukkan pengguna menggunakan *nextDouble()*. Nilai tersebut kemudian disimpan ke dalam variabel *sukuBunga* yang bertipe *double*.

```
input.nextLine();
```

Baris ini kembali digunakan untuk membersihkan karakter *Enter* yang tersisa setelah *nextDouble()*, sehingga input berikutnya dapat dibaca dengan benar.

```
rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
```

Baris ini membuat objek baru dari *class RekeningTabungan*. Data *no*, *nama*, *saldo*, *pin*, dan *sukuBunga* diberikan kepada *constructor RekeningTabungan*. Objek yang telah dibuat kemudian disimpan ke dalam variabel *rekeningBaru*.

```
} else if (produk == 2) {
```

Jika kondisi sebelumnya tidak terpenuhi, program memeriksa apakah pengguna memilih produk 2. Jika produk bernilai 2, program akan menjalankan kode untuk membuat rekening giro.

```
System.out.print("Masukkan Batas Overdraft: ");
```

Baris ini meminta pengguna memasukkan nilai batas *overdraft* untuk rekening giro. Batas *overdraft* merupakan batas penggunaan dana melebihi saldo yang tersedia sesuai aturan yang diterapkan pada rekening giro.

```
double batasOverdraft = input.nextDouble();
```

Baris ini membaca nilai batas *overdraft* yang dimasukkan pengguna dan menyimpannya ke dalam variabel *batasOverdraft* dengan tipe data *double*.

```
input.nextLine();
```

Baris ini digunakan untuk membersihkan karakter *Enter* yang tersisa setelah *nextDouble()*, sama seperti pada bagian input suku bunga sebelumnya.

```
rekeningBaru = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
```

Baris ini membuat objek baru dari *class RekeningGiro*. Nilai *no*, *nama*, *saldo*, *pin*, dan *batasOverdraft* dikirim ke *constructor RekeningGiro*. Objek tersebut kemudian disimpan ke variabel *rekeningBaru*.

```
} else {
```

Bagian else dijalankan apabila nilai produk bukan 1 dan bukan 2.

```
System.out.println("Pilihan produk tidak valid!");
```

Baris ini menampilkan pesan bahwa pilihan produk yang dimasukkan tidak valid.

break;

Baris ini menghentikan proses pada blok atau perulangan tempat kode tersebut berada. Dengan adanya *break*, program tidak akan melanjutkan ke proses penambahan rekening apabila pilihan produk tidak valid.

daftarRekening.add(rekeningBaru);

Baris ini menambahkan objek *rekeningBaru* ke dalam *daftarRekening*. Dengan demikian, rekening yang baru dibuat akan disimpan di dalam *ArrayList* dan dapat digunakan kembali oleh program.

akunAktif = rekeningBaru;

Baris ini menjadikan rekening yang baru dibuat sebagai akun aktif. Artinya, setelah rekening berhasil dibuat, rekening tersebut langsung menjadi rekening yang sedang digunakan oleh pengguna.

System.out.println("Rekening berhasil ditambahkan.");

Baris ini menampilkan pesan kepada pengguna bahwa rekening telah berhasil dibuat dan ditambahkan ke dalam daftar rekening.

break;

Baris ini menghentikan proses pada perulangan atau menu yang sedang berjalan. Setelah rekening berhasil dibuat, program keluar dari proses tersebut sehingga tidak mengulangi pembuatan rekening.

Output

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu: 1
Masukkan No Rekening: 001
Masukkan Nama Pemilik: nisa
Masukkan Saldo Awal: 10000000
Masukkan PIN (6 Digit): 111111
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 1
Suku Bunga (%): 20
Rekening atas nama nisa berhasil dibuat.
Rekening berhasil ditambahkan.
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu: 1
Masukkan No Rekening: 002
Masukkan Nama Pemilik: nisa
Masukkan Saldo Awal: 20000000
Masukkan PIN (6 Digit): 111111
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 2
Masukkan Batas Overdraft: 1000000
Rekening atas nama nisa berhasil dibuat.
Rekening berhasil ditambahkan.
```

2. Simulasi Waktu Akhir Bulan

- Tambahkan menu baru: “7. Simulasi Akhir Bulan (Khusus Tabungan)” di CLI
- Didalam case 7; periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.
- Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
- Jika bukan, tolak dengan pesan: “Gagal: fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.”

a. Class Main

```
case 7:
    if (akunAktif instanceof RekeningTabungan ) {
        RekeningTabungan tab = (RekeningTabungan) akunAktif;
        tab.tambahBungaAkhirBulan();
    } else {
        System.out.println("Gagal: Fitur bunga akhir bulan
        hanya berlaku untuk Rekening Tabungan");
    }
    break;
```

Penjelasan Code:

a. Class Main

case 7:

Baris ini menunjukkan bahwa kode berikutnya dijalankan ketika pengguna memilih menu nomor 7.

if (akunAktif instanceof RekeningTabungan) {

Baris ini mengecek apakah objek yang sedang tersimpan di dalam *akunAktif* merupakan objek dari *class RekeningTabungan*. *instanceof* digunakan untuk mengetahui tipe objek yang sedang digunakan. Jika *akunAktif* memang merupakan rekening tabungan, maka program akan menjalankan kode di dalam *if*.

RekeningTabungan tab = (RekeningTabungan) akunAktif;

Baris ini melakukan casting terhadap *akunAktif* menjadi tipe *RekeningTabungan*, kemudian hasilnya disimpan ke variabel *tab*. *Casting* diperlukan supaya objek tersebut dapat digunakan sebagai *RekeningTabungan* dan *method* khusus seperti *tambahBungaAkhirBulan()* dapat dipanggil.

tab.tambahBungaAkhirBulan();

Baris ini memanggil *method tambahBungaAkhirBulan()* yang terdapat pada *class RekeningTabungan*. *Method* tersebut digunakan untuk menghitung dan menambahkan bunga akhir bulan ke saldo rekening tabungan sesuai dengan aturan yang dibuat di dalam

method tersebut.

} else {

Bagian *else* akan dijalankan apabila *akunAktif* bukan merupakan objek *RekeningTabungan*. Misalnya, akun aktif yang sedang digunakan adalah *RekeningGiro*. Karena fitur bunga akhir bulan khusus untuk rekening tabungan, proses penambahan bunga tidak dilakukan.

System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan");

Baris ini menampilkan pesan kepada pengguna bahwa proses gagal karena fitur bunga akhir bulan hanya dapat digunakan pada rekening tabungan. Jadi kalau akun aktifnya giro, program tidak akan memaksakan bunga ke rekening yang bukan tempatnya. Untuk sekali ini, program punya batasan yang masuk akal.

break;

Baris ini menghentikan proses pada case 7 dan keluar dari struktur switch. Setelah proses bunga selesai atau pesan kesalahan ditampilkan, program akan kembali ke bagian menu utama.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu: 7
Bunga 20.0% berhasil ditambahkan: Rp2000000.0
```

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cek Mutasi (Riwayat)  
7. Simulasi Akhir Bulan (Khusus Tabungan)  
0. Keluar  
Pilih Menu: 7  
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan
```

C. Link Repository GitHub

https://github.com/annisalayli109/PrakPBO_B_2511532024/tree/621423600b97aecd9c2d66009a768977c48ce4f9/PrakPBO_B_2511532024/src/Pekan4_2511532024